Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase VI · Diseño y construcción de optimizadores con IA

Módulo 26

Control óptimo, gemelos digitales y aprendizaje por refuerzo para procesos

PID, identificación de sistemas, control predictivo, SAC y TD3, y puesta en producción en planta

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 26: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$, w, y	estado, entrada manipulada, perturbación y salida medida
$\dot{x} = f(x,u,w)$; $x_{k+1} = f_d(x_k,u_k,w_k)$	modelo en tiempo continuo y en tiempo discreto
A, B, E, C; A_d, B_d	matrices del modelo lineal continuo y del discretizado
Δt, k	periodo de muestreo y índice de instante ($t = k\Delta t$)
$\rho(A)$	radio espectral: mayor módulo de los autovalores de A
r, $e = r - y$	consigna y error de control
K_c, τ_I, τ_D	ganancia, tiempo integral y tiempo derivativo del PID
θ (tema 2)	vector de parámetros de un modelo identificado
N, $\ell(x,u)$, V_f	horizonte, coste de etapa y coste terminal del MPC
$\mathbb{X}$, $\mathbb{U}$	conjuntos de estados y entradas admisibles
$\mathcal{S}$, $\mathcal{A}$, γ, π	estados, acciones, descuento y política (como en el módulo 21)
Q^π, Q_ϕ, π_ψ	función de valor de acción, crítico y actor con parámetros ϕ y ψ
α (tema 4)	temperatura de la entropía en SAC
T, T_{ext}, P, G	temperaturas interior y exterior (°C), potencia eléctrica y ganancias (kW)
R, C (casos de estudio)	resistencia (K/kW) y capacidad (kWh/K) térmicas
COP	coeficiente de rendimiento de la bomba de calor

Índice

Sobre este resumen **ii**

1 Procesos dinámicos y control PID **1**

1.1 Modelos en espacio de estados . . . 1
1.2 Discretización . . . 1
1.3 Estabilidad . . . 2
1.4 El controlador PID . . . 2
1.5 Ajuste del PID . . . 3

2 Identificación de sistemas y gemelos digitales **4**

2.1 El problema de identificación . . . 4
2.2 Modelos no lineales: NARX y redes neuronales . . . 5
2.3 Validación: predicción a un paso y simulación libre . . . 5
2.4 El gemelo digital . . . 6

3 Control predictivo basado en modelo **8**

3.1 Control óptimo de horizonte finito: el regulador lineal cuadrático . . . 8
3.2 El horizonte deslizante . . . 8
3.3 El MPC lineal con restricciones como programa cuadrático . . . 9
3.4 Cálculo en tiempo real y modelos no lineales . . . 10
3.5 MPC con modelo aprendido . . . 10

4 Aprendizaje por refuerzo para control continuo **12**

4.1 Del PPO a los métodos fuera de política . . . 12
4.2 El sesgo de sobreestimación y TD3 . . . 13
4.3 Máxima entropía: SAC . . . 13
4.4 Entornos: la interfaz de Gymnasium . . . 13
4.5 Diseño de la recompensa . . . 14
4.6 RL seguro, restricciones y del simulador a la planta . . . 14

5 Casos de estudio: edificio, reactor, batería y centro de datos **16**

5.1 Cómo comparar controladores . . . 16
5.2 Climatización de un edificio . . . 17
5.3 Control de temperatura de un reactor por lotes . . . 17
5.4 Gestión de una batería con precios horarios . . . 18

5.5 Refrigeración de centros de datos: el caso publicado de Google y DeepMind . . 19

6 Puesta en producción en planta 21

6.1 Arquitectura por capas 21
6.2 Modos de operación y lazo humano 21
6.3 Alarmas 22
6.4 Validación y despliegue 22
6.5 El filtro de seguridad 22
6.6 Ciberseguridad de la tecnología de operación 23
6.7 Normas y regulación 23

Bibliografía 25

Tema 1

Procesos dinámicos y control PID

Antes de optimizar un proceso hay que describirlo como sistema dinámico y entender qué hace con él un controlador realimentado. Este tema presenta los modelos en espacio de estados, su discretización exacta y su estabilidad, siguiendo los capítulos 3 y 6 de Åström y Murray [4], y el controlador PID con sus reglas de ajuste, según el capítulo 11 del mismo libro, el texto de Seborg *et al.* [42], la lección 5 del curso 10.450 del MIT [30] y el método SIMC de Skogestad [43]. El caso de estudio es la zona de un edificio calentada por una bomba de calor, que reaparece en todo el módulo.

1.1 Modelos en espacio de estados

Definición 1.1 (Sistema en espacio de estados). Un *sistema dinámico en espacio de estados* en tiempo continuo es un par de ecuaciones

$$\dot{x}(t) = f\big(x(t), u(t), w(t)\big), \qquad y(t) = h\big(x(t)\big),$$

donde $x(t) \in \mathbb{R}^n$ es el *estado* (la información mínima que, junto con las entradas futuras, determina la evolución futura), $u(t) \in \mathbb{R}^m$ la *entrada manipulada*, $w(t)$ la *perturbación* (entrada que no se elige) e $y(t)$ la *salida medida*. El sistema es *lineal e invariante en el tiempo* (LTI) si $f(x,u,w) = Ax + Bu + Ew$ y $h(x) = Cx$ con matrices constantes.

Los modelos de procesos salen de balances de materia y energía, y casi nunca son lineales. Su comportamiento cerca de un punto de operación se aproxima por uno lineal.

Proposición 1.2 (Linealización). *Sea $(\bar{x}, \bar{u}, \bar{w})$ un* punto de equilibrio*, $f(\bar{x},\bar{u},\bar{w}) = 0$, con f diferenciable con continuidad. Las desviaciones $\delta x = x - \bar{x}$, $\delta u = u - \bar{u}$, $\delta w = w - \bar{w}$ cumplen*

$$\dot{\delta x} = A\,\delta x + B\,\delta u + E\,\delta w + o\big(\|(\delta x, \delta u, \delta w)\|\big),$$

con $A = \partial f/\partial x$, $B = \partial f/\partial u$ y $E = \partial f/\partial w$ evaluados en el equilibrio.

1.2 Discretización

Un controlador digital mide cada Δt y mantiene su salida constante entre dos muestras: es el *retenedor de orden cero* (ZOH). Con esa hipótesis el modelo discreto es exacto.

Proposición 1.4 (Discretización exacta con ZOH). *Sea $\dot{x} = Ax + Bu$ con $u(t) = u_k$ constante en $[k\Delta t, (k+1)\Delta t)$. Entonces $x_{k+1} = A_d x_k + B_d u_k$, con*

$$A_d = e^{A\Delta t}, \qquad B_d = \Big(\int_0^{\Delta t} e^{A\sigma}\, d\sigma\Big) B,$$

y, si A es invertible, $B_d = A^{-1}\big(e^{A\Delta t} - I\big)B$.

La alternativa más sencilla, el método de *Euler explícito*, sustituye la derivada por un cociente incremental: $x_{k+1} = (I + A\Delta t)x_k + B\Delta t\, u_k$. Es solo una aproximación de primer orden, y puede volver inestable un sistema estable (corolario 1.8).

1.3 Estabilidad

Definición 1.6 (Estabilidad asintótica). El sistema discreto $x_{k+1} = A_d x_k$ es *asintóticamente estable* si $x_k \to 0$ para todo estado inicial x_0. Se llama *radio espectral* de A_d a $\rho(A_d) = \max_i |\lambda_i|$, con λ_i sus autovalores (complejos en general).

Teorema 1.7 (Criterio de estabilidad en tiempo discreto). *Sea A_d diagonalizable. El sistema $x_{k+1} = A_d x_k$ es asintóticamente estable si y solo si $\rho(A_d) < 1$.*

El resultado es cierto también sin diagonalizabilidad (con la forma de Jordan), y su análogo continuo exige $\operatorname{Re}\lambda_i(A) < 0$ [4, cap. 6].

Corolario 1.8 (ZOH conserva la estabilidad; Euler no siempre). *Sea A diagonalizable con autovalores reales negativos λ_i. (i) $A_d = e^{A\Delta t}$ tiene autovalores $e^{\lambda_i \Delta t} \in (0,1)$: el modelo ZOH es estable para todo $\Delta t > 0$. (ii) $I + A\Delta t$ tiene autovalores $1 + \lambda_i \Delta t$, y el modelo de Euler es estable si y solo si $\Delta t < 2/\max_i |\lambda_i|$.*

1.4 El controlador PID

Definición 1.10 (Controlador PID). Dado el error $e = r - y$ entre la consigna r y la medida y (figura 1.1), el controlador PID calcula

$$u(t) = K_c\Big(e(t) + \frac{1}{\tau_I}\int_0^t e(s)\,ds + \tau_D\,\frac{de}{dt}\Big),$$

con ganancia K_c, tiempo integral τ_I y tiempo derivativo τ_D. En forma discreta con periodo Δt (forma *posicional*),

$$u_k = K_c e_k + I_k + K_c\tau_D\frac{e_k - e_{k-1}}{\Delta t}, \qquad I_{k+1} = I_k + \frac{K_c \Delta t}{\tau_I}\, e_k.$$

Los tres términos tienen papeles distintos: el proporcional reacciona al error actual, el integral acumula el error pasado y el derivativo anticipa su tendencia. En procesos térmicos lentos y con medidas ruidosas el término derivativo se suele anular (controlador PI) o filtrar [4, cap. 11].

Proposición 1.11 (Error permanente con control proporcional). *Sea una planta de primer orden estable de ganancia estática k y un controlador proporcional $u = \bar{u} + K_c e$ con $kK_c > -1$. Ante un escalón de consigna de amplitud Δr, el error permanente es $e_\infty = \Delta r/(1 + kK_c)$.*

Proposición 1.12 (La acción integral elimina el error permanente). *Sea un lazo con un controlador que contiene la acción integral $I_{k+1} = I_k + (K_c\Delta t/\tau_I)e_k$, con $K_c \neq 0$. Si, con consigna y perturbaciones constantes, el lazo converge a un equilibrio en el que la salida del controlador no está saturada, entonces el error en ese equilibrio es nulo.*

La proposición no dice nada sobre si el lazo converge: eso depende del ajuste. Por ejemplo, con la zona del ejemplo 1.3 ($k = 14$ K/kW) y $K_c = 1{,}43$ kW/K, un control solo proporcional dejaría el $1/(1+20) = 4{,}8$ % del escalón de consigna sin corregir; el PI lo elimina.

Definición 1.13 (Saturación y antisaturación). Todo actuador tiene límites: $u \in [u_{\min}, u_{\max}]$. Si el error persiste mientras la salida está saturada, el término integral crece sin efecto sobre la planta (*windup*) y, cuando el error cambia de signo, el controlador tarda en salir de la saturación, lo que produce sobreoscilaciones. Un mecanismo de *antisaturación* lo evita; el más sencillo es la *integración condicional*: solo se actualiza I_k si la salida no está saturada o si el error tiende a desaturarla [4, cap. 11].

1.5 Ajuste del PID

Definición 1.14 (Modelo de primer orden con retardo). Muchos procesos se aproximan por una respuesta de primer orden con retardo (FOPDT), $y(s)/u(s) = k\,e^{-\theta s}/(\tau s + 1)$: ganancia k, constante de tiempo τ y retardo θ, que pueden leerse de un ensayo en escalón. En un lazo digital, el muestreo y el retenedor añaden un retardo efectivo del orden de $\Delta t/2$ a Δt.

Las reglas de Ziegler y Nichols [52], de 1942, fueron las primeras reglas sistemáticas; producen lazos agresivos y poco amortiguados. Las reglas SIMC de Skogestad [43] parten del FOPDT y de un único parámetro de diseño, la constante de tiempo deseada en lazo cerrado τ_c:

$$K_c = \frac{\tau}{k\,(\tau_c + \theta)}, \qquad \tau_I = \min\bigl\{\tau,\ 4(\tau_c + \theta)\bigr\}. \tag{1.3}$$

La elección $\tau_c = \theta$ da un ajuste «firme» con buen compromiso entre rapidez y robustez; valores mayores de τ_c dan lazos más lentos y robustos. El mínimo en τ_I evita que, en procesos muy lentos como los térmicos, el integrador tarde demasiado en rechazar perturbaciones.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
python-control [14]	funciones de transferencia, espacio de estados, discretización, respuesta en frecuencia, LQR	libre (BSD-3)	análisis y diseño lineal en Python; docencia
`scipy.signal`	sistemas LTI, `cont2discrete`, simulación	libre (BSD-3)	cuando basta con lo esencial y no se quiere otra dependencia
Modelica / OpenModelica [32]	modelado físico acausal por componentes (térmico, eléctrico, fluidos)	lenguaje abierto; entorno libre	modelos de planta complejos y reutilizables
FMI / FMPy [31]	intercambio de modelos (FMU) entre herramientas y cosimulación	estándar abierto; FMPy libre	conectar un modelo de Modelica o de un fabricante con Python
MATLAB/Simulink	diseño de control, autoajuste PID, generación de código	comercial	entornos industriales que ya lo usan
DCS y PLC (IEC 61131-3)	ejecución en planta de los PID con bloques normalizados	comercial	siempre: es donde vive el control básico

Cuadro 1.1: Tecnologías para el modelado y el control clásico.

Tema 2

Identificación de sistemas y gemelos digitales

El MPC necesita un modelo para predecir y el aprendizaje por refuerzo necesita un simulador en el que equivocarse sin coste. En planta casi nunca se dispone de un modelo físico calibrado: hay que construirlo con datos. Este tema formaliza la *identificación de sistemas* siguiendo a Ljung [27], texto del curso 6.435 del MIT [29], extiende los modelos lineales a los no lineales con redes neuronales como en Narendra y Parthasarathy [34] y define el *gemelo digital* según la clasificación de Kritzinger *et al.* [22]. El mensaje central es metodológico: un modelo para control se valida simulando, no prediciendo un paso.

2.1 El problema de identificación

Se dispone de un registro $Z^N = \{(u_k, y_k)\}_{k=0}^{N}$ de entradas y salidas. Se elige una *estructura de modelo*, una familia de predictores $\hat{y}_{k+1}(\theta)$ que dependen de parámetros θ y de los datos hasta el instante k, y se estima θ minimizando el error de predicción [27]:

$$\hat{\theta}_N = \arg\min_{\theta} \; V_N(\theta), \qquad V_N(\theta) = \frac{1}{N}\sum_{k=0}^{N-1} \big(y_{k+1} - \hat{y}_{k+1}(\theta)\big)^2. \tag{2.1}$$

Definición 2.1 (Modelo ARX). Un modelo ARX(n_a, n_b), *autorregresivo con entradas exógenas*, es

$$y_{k+1} = \sum_{i=1}^{n_a} a_i y_{k+1-i} + \sum_{j=1}^{n_b} b_j u_{k+1-j} + \varepsilon_{k+1},$$

con ε ruido blanco. Con el *vector de regresores* $\varphi_k = (y_k, \dots, y_{k+1-n_a}, u_k, \dots, u_{k+1-n_b})$ y $\theta = (a, b)$, se escribe $y_{k+1} = \varphi_k^\top \theta + \varepsilon_{k+1}$: es una regresión lineal.

Proposición 2.2 (Estimador de mínimos cuadrados del ARX). *Sea Φ la matriz cuyas filas son los $\varphi_k^\top$ e Y el vector de los y_{k+1}. Si $\Phi^\top\Phi$ es invertible, el minimizador de (2.1) es $\hat{\theta}_N = (\Phi^\top\Phi)^{-1}\Phi^\top Y$. Si además los datos proceden de un ARX con parámetro θ_0, entonces*

$$\hat{\theta}_N - \theta_0 = \Big(\frac{1}{N}\sum_k \varphi_k\varphi_k^\top\Big)^{-1} \frac{1}{N}\sum_k \varphi_k \varepsilon_{k+1},$$

y $\hat{\theta}_N \to \theta_0$ cuando $\frac{1}{N}\sum \varphi_k\varphi_k^\top$ converge a una matriz invertible y $\frac{1}{N}\sum \varphi_k\varepsilon_{k+1} \to 0$.

La condición $\frac{1}{N}\sum \varphi_k\varepsilon_{k+1} \to 0$ se cumple si ε es blanco, porque φ_k solo contiene datos hasta k. Si el ruido está coloreado (por ejemplo, ruido de medida sobre una salida autorregresiva), y_k y ε_{k+1} están correlacionados y el ARX es sesgado; la solución es estimar también un modelo del ruido (ARMAX, error de salida) o usar variables instrumentales [27]. La condición sobre $\frac{1}{N}\sum \varphi_k\varphi_k^\top$ exige que la entrada sea suficientemente rica.

Definición 2.3 (Excitación persistente). Una señal u es *persistentemente excitante de orden* n si la matriz de covarianzas $\big(R_u(i-j)\big)_{i,j=1}^{n}$, con $R_u(\tau) = \lim_N \frac{1}{N}\sum_k u_k u_{k-\tau}$, es definida positiva.

Una constante es persistentemente excitante solo de orden 1; una sinusoide, de orden 2; un ruido blanco o una secuencia binaria pseudoaleatoria, de cualquier orden. En planta se usan escalones aleatorios de duración variable, que respetan los límites del actuador y excitan las frecuencias que interesan.

2.2 Modelos no lineales: NARX y redes neuronales

Definición 2.5 (Modelo NARX). Un modelo NARX es $y_{k+1} = g(\varphi_k;\theta) + \varepsilon_{k+1}$ con los mismos regresores que el ARX y una función g no lineal: un polinomio, una red de base radial o un perceptrón multicapa (módulo 9). Narendra y Parthasarathy [34] propusieron en 1990 identificar y controlar sistemas dinámicos con redes de este tipo.

Hay tres familias principales, de menos a más estructura física:

(i) *Caja negra*: g es un perceptrón multicapa sobre los regresores. Es flexible, pero solo es fiable dentro del rango de los datos.

(ii) *Modelos de estado neuronales*: un estado latente $h_{k+1} = F(h_k, u_k)$ e $y_k = H(h_k)$ con F y H redes, es decir, una red recurrente (módulo 12, §5.2) entrenada en simulación sobre secuencias, con retropropagación en el tiempo. Captura dinámicas de orden alto sin elegir n_a, a costa de un entrenamiento más delicado.

(iii) *Caja gris*: la física aporta la estructura y los datos los parámetros, o una red aprende solo el residuo del modelo físico. Si se conoce la forma no lineal (por ejemplo, un producto entre entrada y perturbación), basta con añadir el regresor correspondiente y el modelo sigue siendo lineal en θ.

2.3 Validación: predicción a un paso y simulación libre

Definición 2.7 (Dos formas de validar). Sea $\hat f$ un modelo de un paso, $\hat x_{k+1} = \hat f(x_k, u_k)$. La *predicción a un paso* usa en cada instante el estado medido: $\hat x_{k+1} = \hat f(x_k, u_k)$. La *simulación libre* (validación en lazo abierto del modelo, o *free run*) se realimenta con su propia salida: $\tilde x_{k+1} = \hat f(\tilde x_k, u_k)$, $\tilde x_0 = x_0$.

El MPC predice N pasos y el agente de RL se entrena durante episodios enteros en el modelo: ambos usan el modelo en simulación libre. La siguiente proposición explica por qué el error a un paso es engañoso.

Proposición 2.8 (Acumulación del error en simulación). *Sea $x_{k+1} = f(x_k, u_k)$ el sistema real y $\hat f$ un modelo tal que $|\hat f(x,u) - f(x,u)| \le \varepsilon$ para todo (x,u) y $\hat f$ es lipschitziana en x con constante L. Entonces el error de simulación libre cumple*

$$|\tilde x_k - x_k| \le \varepsilon\,\frac{1-L^k}{1-L} \quad (L \neq 1), \qquad \text{y, si } L<1, \quad |\tilde x_k - x_k| \le \frac{\varepsilon}{1-L}.$$

En procesos lentos L es casi 1. Para la zona, $L = a = 0{,}9835$ y $1/(1-L) \approx 61$: un error de modelo de 0,005 K por paso, invisible a un paso, puede convertirse en 0,30 K en simulación, y en 0,24 K tras las 96 muestras de un horizonte de 24 h.

Proposición 2.9 (Identificabilidad en lazo cerrado). *Sea $y_{k+1} = ay_k + bu_k + \varepsilon_{k+1}$ y supóngase que los datos se registran con un controlador proporcional $u_k = -Ky_k$ y sin excitación externa. Entonces $\Phi^\top\Phi$ es singular y todos los pares (a', b') con $a' - Kb' = a - Kb$ dan el mismo error de predicción: a y b no son identificables por separado.*

Es la razón por la que no basta con descargar el histórico de un lazo que funciona bien: la entrada es una función de la salida y no aporta información independiente. Las soluciones son añadir excitación (escalones o ruido en la consigna o en la salida del controlador), alternar entre controladores distintos o registrar periodos en manual [27]. El histórico de un PID también es poco informativo por otra razón: el controlador mantiene la planta cerca de un único punto de operación. Lazic *et al.* [23] lo comprobaron en un centro de datos: tres horas de exploración aleatoria y acotada produjeron un modelo mejor que una semana de datos históricos de los PID (tema 5).

Observación 2.10 (Partición temporal de los datos). Las muestras de una serie temporal no son independientes: barajarlas antes de separar entrenamiento y validación (módulo 8, §1.4) filtra información, porque cada muestra de validación tiene vecinas casi idénticas en el entrenamiento. Se separan *bloques de tiempo* (días o semanas completos) y se reserva el último bloque, con condiciones distintas (otra estación del año, otro régimen de producción), como prueba final.

2.4 El gemelo digital

Definición 2.11 (Modelo, sombra y gemelo digital). Kritzinger *et al.* [22] distinguen tres niveles según la automatización del flujo de datos entre el objeto físico y su representación digital: un *modelo digital* intercambia datos con la planta solo de forma manual; una *sombra digital* recibe automáticamente los datos de la planta pero no actúa sobre ella; un *gemelo digital* intercambia datos automáticamente en los dos sentidos.

Para control, el gemelo tiene cuatro componentes (figura 2.1): un *modelo* (físico, de datos o híbrido); un *estimador de estado*, porque no todos los estados se miden (el filtro de Kalman o la estimación de horizonte móvil, tema 3); un procedimiento de *recalibración* cuando los residuos derivan, y una *interfaz* estándar para que lo usen otros programas: el estándar FMI [31], que empaqueta un modelo como unidad intercambiable (FMU), o la interfaz de entornos de Gymnasium [49]. Para edificios, el simulador EnergyPlus [7] y bibliotecas de Modelica [32] son la referencia física, y marcos como BOPTEST [5] o Sinergym [6] los envuelven como entornos de prueba para controladores.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
NumPy / SciPy	mínimos cuadrados, ARX, filtrado	libre (BSD-3)	modelos lineales en los parámetros
SysIdentPy	modelos NARMAX, selección de regresores	libre (BSD-3)	NARX polinómicos interpretables
scikit-learn	perceptrones, *ridge*, validación	libre (BSD-3)	NARX neuronales pequeños
PyTorch	redes recurrentes, modelos de estado neuronales, entrenamiento en simulación	libre (tipo BSD)	modelos neuronales con dinámica latente o híbridos con física
MATLAB System Identification Toolbox	ARX, ARMAX, error de salida, subespacios	comercial	entornos ya basados en MATLAB
FMI / FMPy, Modelica	encapsular y ejecutar el gemelo	estándar abierto / libre	conectar el gemelo con otras herramientas
EnergyPlus [7]	simulación energética detallada de edificios	libre (tipo BSD)	gemelo físico de un edificio; generar datos

Cuadro 2.1: Tecnologías para la identificación y los gemelos digitales.

Tema 3

Control predictivo basado en modelo

El PID corrige el error presente; no sabe que mañana a las 10 la electricidad será cara, ni que la potencia de la bomba de calor está limitada. El *control predictivo basado en modelo* (MPC) usa un modelo para predecir el futuro, resuelve en cada instante un problema de optimización con restricciones sobre un horizonte y aplica solo la primera decisión. Este tema sigue los capítulos 1 y 2 de Rawlings, Mayne y Diehl [39], las clases 16 («Linear quadratic regulator: Discrete-time finite horizon») a 18 de EE363 de Stanford [44] y la clase 10 de CS285 [24], que presenta el control óptimo y la planificación como punto de partida del RL basado en modelos. La aplicación a edificios sigue la revisión de Drgoňa *et al.* [9].

3.1 Control óptimo de horizonte finito: el regulador lineal cuadrático

Definición 3.1 (Problema de control óptimo de horizonte finito). Dados el modelo $x_{k+1} = f(x_k, u_k)$, un coste de etapa $\ell(x,u)$, un coste terminal $V_f(x)$, conjuntos admisibles $\mathbb{X}$, $\mathbb{U}$, $\mathbb{X}_f$ y un estado inicial x, el problema de horizonte N es

$$\mathbb{P}_N(x): \quad V_N^0(x) = \min_{u_0,\dots,u_{N-1}} \sum_{j=0}^{N-1} \ell(x_j,u_j) + V_f(x_N)$$
$$\text{s.a.} \quad x_0 = x, \quad x_{j+1} = f(x_j,u_j), \quad x_j \in \mathbb{X}, \quad u_j \in \mathbb{U}, \quad x_N \in \mathbb{X}_f.$$

Sin restricciones, con modelo lineal y costes cuadráticos, el problema tiene solución explícita.

Teorema 3.2 (LQR de horizonte finito). *Sean $x_{k+1} = Ax_k + Bu_k$, $\ell(x,u) = x^\top Q x + u^\top R u$ y $V_f(x) = x^\top P_f x$, con $Q \succeq 0$, $P_f \succeq 0$ y $R \succ 0$, sin restricciones. Se define hacia atrás $P_N = P_f$ y, para $k = N-1, \dots, 0$,*

$$K_k = (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A, \qquad P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B\, K_k.$$

Entonces el coste óptimo desde x en el instante k es $V_k(x) = x^\top P_k x$ y la decisión óptima es $u_k = -K_k x_k$.

3.2 El horizonte deslizante

Definición 3.4 (Control predictivo). En cada instante k se mide (o se estima) el estado x_k, se resuelve $\mathbb{P}_N(x_k)$, se aplica la primera decisión $u_k = u_0^*(x_k)$, se descarta el resto y se repite en $k+1$. La ley de control implícita $\kappa_N(x) = u_0^*(x)$ es una realimentación del estado.

La realimentación procede de volver a resolver con cada nueva medida: si una perturbación desvía la planta de la predicción, el siguiente problema parte del estado real. Que la ley resultante estabilice la planta no es automático, porque el controlador optimiza un horizonte finito que se desplaza. El resultado clásico pide unos ingredientes terminales [39, cap. 2].

Teorema 3.5 (Factibilidad recursiva y estabilidad). *Supóngase que $f(0,0)=0$, que $\ell(x,u)\geq c\|x\|^2$ con $c>0$, y que existen un conjunto terminal $\mathbb{X}_f\subseteq\mathbb{X}$ y una ley local κ_f tales que, para todo $x\in\mathbb{X}_f$: $\kappa_f(x)\in\mathbb{U}$, $f(x,\kappa_f(x))\in\mathbb{X}_f$ y $V_f\big(f(x,\kappa_f(x))\big)-V_f(x)\leq-\ell\big(x,\kappa_f(x)\big)$. Si $\mathbb{P}_N(x_0)$ es factible, entonces: (i) $\mathbb{P}_N(x_k)$ es factible para todo k; (ii) $V_N^0(x_{k+1})\leq V_N^0(x_k)-\ell\big(x_k,\kappa_N(x_k)\big)$, y (iii) $x_k\to 0$.*

En el caso lineal-cuadrático, la elección habitual es $V_f(x)=x^\top Px$ con P la solución de la ecuación de Riccati de horizonte infinito y $\kappa_f=-Kx$ el LQR, y $\mathbb{X}_f$ un conjunto en el que el LQR no viola ninguna restricción. En la práctica industrial se usan a menudo horizontes largos sin ingredientes terminales explícitos, y la estabilidad se comprueba por simulación.

3.3 El MPC lineal con restricciones como programa cuadrático

Con modelo lineal, la trayectoria predicha es lineal en las decisiones. Sean $U=(u_0,\dots,u_{N-1})$ y $X=(x_1,\dots,x_N)$. Por recursión, $x_j=A^jx_0+\sum_{i<j}A^{j-1-i}Bu_i$, es decir,

$$X=\Phi x_0+\Gamma U,\qquad \Phi=\begin{pmatrix}A\\A^2\\\vdots\\A^N\end{pmatrix},\qquad \Gamma=\begin{pmatrix}B&0&\cdots&0\\AB&B&\cdots&0\\\vdots&&\ddots&\\A^{N-1}B&A^{N-2}B&\cdots&B\end{pmatrix}. \tag{3.1}$$

Proposición 3.6 (El MPC lineal-cuadrático es un QP estrictamente convexo). *Sean $\bar Q=\operatorname{diag}(Q,\dots,Q,P_f)\succeq 0$ y $\bar R=\operatorname{diag}(R,\dots,R)\succ 0$. El coste $\sum_{j<N}(x_j^\top Qx_j+u_j^\top Ru_j)+x_N^\top P_fx_N$ vale*

$$J(U)=U^\top HU+2x_0^\top F^\top U+x_0^\top(Q+\Phi^\top\bar Q\Phi)x_0,\qquad H=\Gamma^\top\bar Q\Gamma+\bar R,\quad F=\Gamma^\top\bar Q\Phi,$$

y $H\succ 0$. Con restricciones lineales sobre U y X (límites de actuadores, de velocidad de cambio, de estados), el problema es un programa cuadrático estrictamente convexo con, a lo sumo, una solución.

Definición 3.8 (Restricciones blandas). Una restricción de estado $g(x_j)\leq 0$ puede volver infactible el problema si una perturbación saca la planta del conjunto admisible. Se *ablanda* con una holgura $s_j\geq 0$: $g(x_j)\leq s_j$, y se añade $\rho\sum_j s_j$ al coste.

Proposición 3.9 (Penalización exacta). *Sea el problema convexo $\min f(z)$ s.a. $g_i(z)\leq 0$, con solución z^* y multiplicadores de Karush–Kuhn–Tucker $\lambda_i^*\geq 0$. Si $\rho\geq\max_i\lambda_i^*$, entonces z^* minimiza también $f(z)+\rho\sum_i\max\{g_i(z),0\}$, que es la versión con holguras óptimas.*

Es la versión convexa del teorema 17.3 de Nocedal y Wright [36, §17.2]. En la práctica: si ρ es suficientemente grande, las holguras solo se activan cuando el problema con restricciones duras es infactible; si es pequeño, el optimizador «compra» violaciones de confort cuando le salen baratas. En el ejemplo 3.7, con la restricción $u_0\geq -0{,}5$ ablandada, cualquier $\rho\geq 0{,}5$ reproduce la solución dura.

Definición 3.10 (MPC económico). Cuando el coste de etapa es un coste económico (euros de energía, producción, emisiones) en lugar de una desviación respecto a una consigna, se habla de *MPC económico*. Si el modelo es lineal y el coste y las restricciones son lineales, el problema es un programa lineal.

3.4 Cálculo en tiempo real y modelos no lineales

Cada problema debe resolverse en una fracción del periodo de muestreo. Los QP de MPC lineal se resuelven con métodos de punto interior, de conjunto activo o de separación de operadores como OSQP [46], que se benefician de arrancar desde la solución anterior desplazada (*warm start*). Con modelo no lineal, $\mathbb{P}_N$ es un programa no lineal (NMPC) y no convexo en general. Las técnicas estándar [39, cap. 8] son el *disparo múltiple* (los estados de cada paso son también variables, con la dinámica como restricción de igualdad, lo que mejora el condicionamiento) y la *iteración en tiempo real*: un solo paso de programación cuadrática secuencial por instante, que aprovecha que el problema cambia poco de un instante al siguiente. CasADi [3] proporciona la diferenciación automática y la formulación; acados [50] y do-mpc [12] construyen sobre ellas resolvedores rápidos y herramientas completas de MPC y de *estimación de horizonte móvil*, que es el problema dual de estimar el estado a partir de las últimas medidas.

3.5 MPC con modelo aprendido

El modelo del MPC puede ser el identificado en el tema 2:

(i) *Modelo lineal en las entradas* (ARX o NARX lineal en u con coeficientes que dependen de perturbaciones previstas): el MPC sigue siendo un QP o un LP. Es la opción preferida cuando basta.

(ii) *Red neuronal*: $\hat{f}_\theta$ entra en las restricciones de igualdad del NLP. Los gradientes que necesita el optimizador son productos vector-jacobiano de la red, que proporciona la retropropagación (módulo 10, §5.4, donde se ve que los adjuntos son multiplicadores de Lagrange de la dinámica). Lazic *et al.* [23] optimizaron así un MPC con modelo ARX directamente con Adam (módulo 7, §5.3) en TensorFlow, parametrizando las acciones con una tangente hiperbólica para respetar sus límites.

(iii) *Robustez frente al error de modelo*: la proposición 2.8 permite *endurecer* las restricciones con un margen que crece con el horizonte, $\underline{T} + \varepsilon(1 - L^j)/(1 - L)$ en el paso j; un margen constante es la versión más simple.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
CVXPY [8]	lenguaje de modelado de problemas convexos; parámetros para resolver repetidamente	libre (Apache-2.0)	MPC lineal (LP, QP) en Python y prototipos
OSQP [46], Clarabel, HiGHS	resolvedores de QP, cónicos y de LP	libres (Apache-2.0, MIT)	bajo CVXPY o embebidos
CasADi [3]	diferenciación automática simbólica, NLP, integradores, interfaz a IPOPT	libre (LGPL-3)	NMPC y estimación con modelos físicos o neuronales
acados [50]	NMPC y estimación en tiempo real (iteración en tiempo real, HPIPM), generación de código C	libre (BSD-2)	sistemas embebidos y muestreo rápido
do-mpc [12]	MPC no lineal, robusto multietapa y estimación de horizonte móvil sobre CasADi	libre (LGPL-3)	prototipos completos de NMPC en Python
Pyomo	modelado algebraico de LP, MILP y NLP	libre (BSD-3)	planificación con variables enteras
MPC comerciales de los fabricantes de DCS	MPC multivariable industrial integrado con el sistema de control	comercial	refinerías y plantas químicas con soporte del fabricante

Cuadro 3.1: Tecnologías para el control predictivo.

Tema 4

Aprendizaje por refuerzo para control continuo

El módulo 21 formuló el aprendizaje por refuerzo para modelos de lenguaje: acciones discretas (tokens), recompensa al final y PPO, un método *en política* que descarta las muestras tras cada actualización. En control de procesos las acciones son continuas (potencias, aperturas de válvula), las recompensas llegan en cada paso y cada muestra cuesta: viene de un simulador lento o de la planta. Este tema presenta los métodos actor-crítico *fuera de política* que reutilizan la experiencia (DDPG, TD3 y SAC), cómo se empaqueta un proceso como entorno, cómo se diseña la recompensa sin cambiar el problema y cómo se tratan la seguridad y el salto del simulador a la planta. Sigue a Sutton y Barto [47], las clases 7 («Value Function Methods»), 8 («Deep RL with Q-Functions») y 10 de CS285 [24], la tarea 3 de ese curso [25] y los artículos originales citados en cada sección.

4.1 Del PPO a los métodos fuera de política

Definición 4.1 (MDP de control continuo). Un MDP de control continuo tiene estados $s \in \mathcal{S} \subseteq \mathbb{R}^n$, acciones $a \in \mathcal{A} = [a_{\min}, a_{\max}]^m$, dinámica $s' \sim p(\cdot \mid s, a)$, recompensa $r(s, a, s')$ y descuento $\gamma < 1$. La función de valor de acción de una política π cumple la ecuación de Bellman (módulo 21, tema 1)

$$Q^\pi(s,a) = \mathbb{E}_{s'}\big[r(s,a,s') + \gamma\, \mathbb{E}_{a' \sim \pi(\cdot \mid s')} Q^\pi(s', a')\big].$$

PPO sirve para acciones continuas con una política gaussiana $\pi_\psi(a \mid s) = \mathcal{N}(\mu_\psi(s), \operatorname{diag}\sigma^2)$ y es robusto, pero cada lote de experiencia se usa para unas pocas épocas y se descarta. Los métodos fuera de política guardan las transiciones (s, a, r, s', d) en un *búfer de repetición* y aprenden de ellas muchas veces, porque la ecuación de Bellman para Q^π se puede imponer sobre transiciones generadas por cualquier política. El precio es la inestabilidad del aprendizaje de Q con aproximadores no lineales, que se mitiga con *redes objetivo*: copias lentas $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$, con $\tau \approx 0{,}005$, que fijan los objetivos de regresión [24].

Definición 4.2 (DDPG). DDPG [26] aprende un crítico $Q_\phi(s,a)$ y un actor determinista $a = \mu_\psi(s)$. El crítico minimiza $\mathbb{E}\big[(Q_\phi(s,a) - y)^2\big]$ sobre el búfer con el objetivo $y = r + \gamma(1-d)\, Q_{\phi'}\big(s', \mu_{\psi'}(s')\big)$, y el actor asciende por el gradiente $\nabla_\psi \mathbb{E}_s\big[Q_\phi(s, \mu_\psi(s))\big] = \mathbb{E}_s\big[\nabla_a Q_\phi(s,a)|_{a=\mu_\psi(s)} \nabla_\psi \mu_\psi(s)\big]$, por la regla de la cadena (módulo 4, tema 2). La exploración se hace sumando ruido a la acción.

4.2 El sesgo de sobreestimación y TD3

Proposición 4.3 (Sobreestimación por maximización). *Sean $\hat{Q}(a)$, $a \in \mathcal{A}$ finito, estimaciones aleatorias con $\mathbb{E}[\hat{Q}(a)] = Q(a)$. Entonces $\mathbb{E}\big[\max_a \hat{Q}(a)\big] \geq \max_a Q(a)$. Si son independientes, gaussianas de varianza σ^2 y todos los $Q(a)$ son iguales a q, con dos acciones $\mathbb{E}[\max_a \hat{Q}(a)] = q + \sigma/\sqrt{\pi}$.*

El objetivo de Bellman contiene un máximo, explícito en Q-learning o implícito en el actor que busca la acción de mayor Q_ϕ; el error de estimación se convierte en sesgo positivo y se propaga hacia atrás por el *bootstrapping*. Con diez acciones de igual valor y $\sigma = 1$, el sesgo de cada objetivo es de 1,54. Sutton y Barto lo llaman *sesgo de maximización* y lo corrigen con el doble aprendizaje [47, §6.7].

Definición 4.4 (TD3). TD3 [13] modifica DDPG con tres mecanismos: (i) *doble Q recortado*: dos críticos Q_{ϕ_1}, Q_{ϕ_2} y el objetivo $y = r + \gamma(1-d)\min_{i=1,2} Q_{\phi_i'}(s', \tilde{a}')$, que sustituye el sesgo positivo por uno ligeramente negativo; (ii) *suavizado de la política objetivo*: $\tilde{a}' = \mathrm{clip}\big(\mu_{\psi'}(s') + \mathrm{clip}(\epsilon, -c, c), a_{\min}, a_{\max}\big)$ con $\epsilon \sim \mathcal{N}(0, \tilde{\sigma}^2)$, que impide explotar picos estrechos y erróneos del crítico; (iii) *actualizaciones retardadas*: el actor y las redes objetivo se actualizan una vez por cada dos actualizaciones de los críticos.

4.3 Máxima entropía: SAC

Definición 4.5 (Objetivo de máxima entropía). SAC [16] maximiza $J(\pi) = \sum_t \mathbb{E}\big[r_t + \alpha\,\mathcal{H}(\pi(\cdot \mid s_t))\big]$, con $\mathcal{H}$ la entropía y $\alpha > 0$ la *temperatura*. Las funciones de valor «blandas» cumplen

$$Q(s,a) = \mathbb{E}_{s'}\big[r + \gamma V(s')\big], \qquad V(s) = \mathbb{E}_{a\sim\pi}\big[Q(s,a) - \alpha \log \pi(a \mid s)\big].$$

Proposición 4.6 (Política blanda óptima). *Para un estado con acciones finitas y valores $Q(a)$, la política que maximiza $\mathbb{E}_\pi[Q] + \alpha\mathcal{H}(\pi)$ es $\pi^*(a) = \exp(Q(a)/\alpha)/Z$, con $Z = \sum_a \exp(Q(a)/\alpha)$, y el valor máximo es $V = \alpha \log Z$.*

En acciones continuas, el actor de SAC es una gaussiana comprimida, $a = \tanh(\mu_\psi(s) + \sigma_\psi(s) \odot \xi)$ con $\xi \sim \mathcal{N}(0, I)$, escalada al rango de la acción. Esta *reparametrización* permite derivar $\mathbb{E}[Q_\phi(s,a) - \alpha \log \pi_\psi(a \mid s)]$ respecto a ψ a través de la muestra, y el cambio de variable corrige la densidad: $\log \pi(a \mid s) = \log \mathcal{N}(u; \mu, \sigma^2) - \sum_i \log(1 - \tanh^2 u_i)$. SAC usa además dos críticos con el mínimo, como TD3. La versión posterior [17] ajusta α automáticamente para mantener una entropía objetivo $\bar{\mathcal{H}}$ (típicamente $-\dim \mathcal{A}$), descendiendo por $\mathbb{E}[-\alpha \log \pi(a \mid s) - \alpha\bar{\mathcal{H}}]$: si la entropía de la política cae por debajo del objetivo, α sube (ejercicio 4.5). La tarea 3 de CS285 [25] implementa estos componentes paso a paso.

4.4 Entornos: la interfaz de Gymnasium

Definición 4.8 (Entorno Gymnasium). Un entorno de Gymnasium [49] es un objeto con un espacio de observaciones, un espacio de acciones (por ejemplo, `Box` para vectores continuos acotados) y dos métodos: `reset(seed)`, que devuelve `(obs, info)`, y `step(accion)`, que devuelve `(obs, recompensa, terminated, truncated, info)`. El indicador `terminated` señala un estado terminal del MDP (el valor futuro es nulo) y `truncated` un corte externo, como un límite de tiempo (el valor futuro no es nulo, simplemente no se observa).

La distinción importa al calcular objetivos: el término $(1-d)$ de DDPG, TD3 y SAC debe usar $d = \texttt{terminated}$, nunca `truncated`. Un episodio de climatización que se corta a los tres días no termina el mundo: si se tratara como terminal, el último objetivo sería $y = r$ en lugar de $r+\gamma V(s')$, y el agente aprendería que justo antes del corte da igual pasar frío (ejercicio 4.1).

Con la interfaz estándar, entrenar un agente es una línea con Stable-Baselines3 [38], que implementa PPO, SAC, TD3 y otros algoritmos:

```
from stable_baselines3 import SAC
modelo = SAC("MlpPolicy", EntornoHVAC(), learning_rate=3e-4,
             gamma=0.99, seed=0)
modelo.learn(total_timesteps=50_000)
```

Las recomendaciones prácticas de esa biblioteca coinciden con la experiencia en procesos: normalizar observaciones y acciones (preferiblemente a $[-1,1]$), usar varias semillas y evaluar con un conjunto de episodios distinto del de entrenamiento.

4.5 Diseño de la recompensa

En control de procesos la recompensa suele ser el negativo de un coste económico más penalizaciones: $r_k = -\big(c_k P_k \Delta t + \rho\, v_{k+1}\Delta t\big)$, con v la violación del confort, como en el MPC del ejemplo 3.12. Usar el mismo coste en MPC y en RL es lo que hace comparables los dos enfoques. Dos transformaciones de la recompensa merecen atención: sumar una constante y sumar términos de *conformación* (*shaping*) para guiar el aprendizaje.

Teorema 4.9 (Conformación basada en potenciales). *Sea $\Phi : \mathcal{S} \to \mathbb{R}$ acotada y $F(s,a,s') = \gamma\Phi(s') - \Phi(s)$. En un MDP con $\gamma < 1$, las funciones de valor de cualquier política π con la recompensa $r' = r + F$ cumplen $Q'^{\pi}(s,a) = Q^{\pi}(s,a) - \Phi(s)$. En consecuencia, las políticas óptimas con r y con r' coinciden.*

El resultado es de Ng, Harada y Russell [35], que demostraron además que los potenciales son, en un sentido preciso, la única forma de conformación que garantiza no alterar la política óptima. Una bonificación arbitraria, como premiar estar cerca de la consigna, puede crear ciclos que el agente explota indefinidamente. Sumar una constante a todas las recompensas es inocuo en tareas continuadas, pero no en episódicas con terminación, donde altera el valor relativo de terminar pronto o tarde [47, ejercicios 3.15 y 3.16].

4.6 RL seguro, restricciones y del simulador a la planta

Definición 4.10 (MDP con restricciones). Un MDP con restricciones (CMDP) [2] añade costes $c(s,a)$ y límites d: $\max_\pi J_r(\pi)$ s.a. $J_c(\pi) \le d$, con J_r y J_c los valores esperados descontados de la recompensa y del coste. Su lagrangiana es $L(\pi,\lambda) = J_r(\pi) - \lambda\big(J_c(\pi) - d\big)$, con $\lambda \ge 0$.

Proposición 4.11 (Dualidad débil). *Para todo $\lambda \ge 0$, $\max_\pi L(\pi,\lambda) \ge J_r(\pi^\star)$, con $\pi^\star$ óptima del CMDP. Por tanto $\min_{\lambda\ge 0}\max_\pi L(\pi,\lambda)$ es una cota superior del valor óptimo.*

Los métodos lagrangianos alternan una mejora de la política con λ fijo, con cualquier algoritmo de RL sobre $r - \lambda c$, y un paso de ascenso dual, $\lambda \leftarrow \max\{0, \lambda + \eta(\hat{J}_c - d)\}$: si la

restricción se viola, el coste se encarece. Garantizan el cumplimiento en media al final del entrenamiento, no durante él. Los métodos de región de confianza como CPO [1] buscan mantener la factibilidad en cada actualización. En planta, ninguna de estas garantías estadísticas sustituye a un *filtro de seguridad* determinista entre el agente y el actuador (tema 6).

Un agente entrenado en un gemelo digital sufre el *salto de la simulación a la realidad*. El siguiente resultado acota su efecto.

Teorema 4.12 (Lema de simulación). *Sean dos MDP con los mismos estados (finitos), acciones y recompensas,* $|r| \le r_{\max}$*, y dinámicas* p *y* $\hat{p}$ *con* $\max_{s,a} \|p(\cdot \mid s,a) - \hat{p}(\cdot \mid s,a)\|_1 \le \epsilon$*. Para toda política* π*,* $\|V^\pi - \hat{V}^\pi\|_\infty \le \gamma\epsilon\, r_{\max}/(1-\gamma)^2$*.*

El factor $1/(1-\gamma)^2$ dice que los errores de modelo pesan más cuanto más largo es el horizonte efectivo: es la versión estocástica de la proposición 2.8. Dos remedios complementarios son mejorar el modelo (identificación, tema 2) y hacer la política robusta a lo que no se sabe del modelo con la *aleatorización del dominio*: entrenar maximizando $\mathbb{E}_{\xi \sim p(\xi)}[J(\pi;\xi)]$, donde ξ son los parámetros inciertos del simulador. Tobin *et al.* [48] aleatorizaron la apariencia visual del simulador y Peng *et al.* [37], su dinámica, y en ambos casos los modelos entrenados solo en simulación funcionaron en robots reales: un detector de objetos usado para agarrar, en el primero, y una política de empuje de objetos, en el segundo. Cuando no hay simulador fiable pero sí históricos, el RL *fuera de línea* aprende de un conjunto fijo de datos (CS285, clases 15 y 16 de otoño de 2023 [24]).

Observación 4.13 (Búsqueda de política sin gradiente). Para políticas con pocos parámetros, la búsqueda aleatoria o el método de entropía cruzada (CEM), que muestrean parámetros, evalúan episodios completos y se quedan con los mejores, son referencias sorprendentemente competitivas: Mania, Guy y Recht [28] obtuvieron con búsqueda aleatoria de políticas lineales resultados comparables a los de métodos profundos en tareas de locomoción. Son fáciles de paralelizar, no necesitan crítico y solo usan el retorno, lo que las hace robustas a recompensas dispersas.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
Gymnasium [49]	interfaz estándar de entornos, envoltorios, comprobador de entornos	libre (MIT)	siempre que se vaya a entrenar un agente
Stable-Baselines3 [38]	PPO, SAC, TD3, DQN fiables y documentados	libre (MIT)	primera opción para entrenar y comparar
CleanRL	implementaciones de un solo archivo	libre (MIT)	entender o modificar un algoritmo
RLlib (Ray), TorchRL	entrenamiento distribuido, muchos entornos en paralelo	libres (Apache-2.0, MIT)	simuladores lentos y grandes presupuestos de cómputo
Safety-Gymnasium	entornos con costes y restricciones	libre (Apache-2.0)	evaluar métodos de RL seguro
Sinergym [6], BOPTEST [5]	edificios simulados (EnergyPlus, Modelica) como entornos y bancos de prueba	libres	control de climatización con modelos físicos detallados

Cuadro 4.1: Tecnologías para el aprendizaje por refuerzo en control.

Tema 5

Casos de estudio: edificio, reactor, batería y centro de datos

Este tema reúne los métodos de los anteriores en cuatro problemas de distinta naturaleza: un edificio, donde el objetivo es económico y las restricciones son de confort; un reactor por lotes, donde lo que manda es la seguridad; una batería, donde el problema es casi puramente de optimización, y un centro de datos, el caso industrial publicado más conocido de aprendizaje automático aplicado a la operación de una instalación. Antes se fija cómo comparar controladores de forma honesta. Las fuentes son la revisión de Drgoňa *et al.* [9] y el marco BOPTEST [5] para edificios, la lección 6 de MIT 10.450 [30] y el texto de Seborg *et al.* [42] para el reactor, y las publicaciones de Google y DeepMind [11, 15] y de Lazic *et al.* [23] para los centros de datos. Los resultados numéricos de los tres primeros casos proceden del cuaderno del módulo.

5.1 Cómo comparar controladores

Definición 5.1 (Indicadores de rendimiento). Para un periodo de evaluación de n pasos se definen la energía $E = \Delta t \sum_k P_k$ (kWh), el coste $\Sigma = \Delta t \sum_k c_k P_k$ (€), la violación del confort $D = \Delta t \sum_k \big(\max\{\underline{T}_k - T_k, 0\} + \max\{T_k - \overline{T}_k, 0\}\big)$ (K · h) y, si se quiere un único número, el coste total $\Sigma + \rho D$ con el mismo ρ que usan los controladores.

Un protocolo de comparación justo exige: (i) el mismo periodo de prueba, no usado para ajustar ningún controlador; (ii) la misma información, en particular las mismas previsiones: un MPC con la meteorología observada frente a un PID que no la conoce compara información, no métodos; (iii) un controlador de referencia bien ajustado, no un PID descuidado; (iv) los mismos ruidos, y (v) varios periodos o semillas, con una medida de la dispersión. BOPTEST [5] se creó precisamente para normalizar estas comparaciones en edificios: fija los modelos, los escenarios y los indicadores.

Proposición 5.2 (Números aleatorios comunes). *Sean X e Y los costes de dos controladores en un mismo escenario aleatorio, con varianzas σ_X^2, σ_Y^2 y correlación r. Entonces* $\operatorname{Var}(X - Y) = \sigma_X^2 + \sigma_Y^2 - 2r\sigma_X\sigma_Y$. *Si $\sigma_X = \sigma_Y = \sigma$, evaluar los dos controladores con los mismos escenarios ($r > 0$) divide la varianza de la diferencia por $1 - r$ respecto a usar escenarios independientes ($r = 0$).*

El coste semanal de climatización depende sobre todo del tiempo meteorológico: dos controladores evaluados en la misma semana tienen costes muy correlacionados, y su diferencia se estima con mucha menos incertidumbre que cada coste por separado. El mismo principio guía el entrenamiento del agente del ejemplo 4.15, que evalúa todos los candidatos de una iteración en los mismos episodios.

5.2 Climatización de un edificio

El caso de la zona ha recorrido los temas 1 a 4. El cuadro 5.1 reúne los resultados en la semana de prueba (temperatura exterior media de 3,35 °C) con la planta nominal y con una planta perturbada (R un 20 % mayor, C un 20 % menor y COP un 10 % peor) que ninguno de los controladores conoce.

	Planta nominal				Planta perturbada	
Controlador	E (kWh)	Σ (€)	D (K · h)	Ahorro	Σ (€)	D (K · h)
PI	175,4	21,06	0,00	—	18,75	0,00
MPC, margen 0,3 K	171,1	19,28	0,00	8,4 %	17,28	0,00
MPC, sin margen	166,9	18,76	3,54	—	—	—
RL (CEM)	169,9	20,63	0,01	2,1 %	18,50	0,00

Cuadro 5.1: Climatización de la zona: comparación en la semana de prueba (cuaderno, Partes 2, 4, 6 y 7): energía E, coste Σ y violación del confort D (definición 5.1). PI con ajuste SIMC y consigna programada; MPC económico; agente CEM con aleatorización del dominio. El ahorro es en coste respecto al PI.

Los resultados ilustran rasgos generales que la literatura sobre edificios documenta con casos reales [9, 51]:

(i) *El MPC gana porque planifica*: conoce el precio y las ganancias de las próximas 24 h y usa la masa térmica para desplazar el consumo de punta a valle. Su ventaja se mantiene con la planta perturbada (un 7,8 % menos de coste que el PI), porque realimenta el estado medido en cada paso.

(ii) *El margen es parte del diseño*: sin él, el MPC «ahorra» un 2,7 % más a costa de violar el confort.

(iii) *El agente aprende sin modelo explícito, pero con un simulador*: con una política lineal de 8 parámetros mejora al PI y es robusto a la perturbación de la planta gracias a la aleatorización, pero queda lejos del MPC, que dispone de previsiones y de un horizonte de planificación explícito. Una política más rica (una red con previsiones como entrada) y un algoritmo como SAC pueden acortar la distancia, con más datos de simulación y más riesgo de explotar defectos del gemelo.

(iv) *El PI sigue siendo la referencia*: no necesita modelo, ni previsiones, ni entrenamiento, y en este caso solo pierde un 8 % de coste. En la práctica, el MPC o el agente se colocan *encima* de los PI existentes, enviándoles consignas (tema 6).

5.3 Control de temperatura de un reactor por lotes

En un reactor por lotes con una reacción exotérmica, la temperatura sigue una receta (calentar, mantener, enfriar) y el riesgo dominante es el *embalamiento térmico*: si el calor generado por la reacción crece con la temperatura más deprisa de lo que la camisa puede retirarlo, la temperatura se dispara. El análisis de estabilidad de la lección 6 de MIT 10.450 [30], que compara las curvas de calor generado y calor retirado, se resume en el criterio siguiente.

	PID	MPC	RL
Necesita	ajuste local (FOPDT)	modelo identificado y previsiones	simulador fiable (gemelo) y entrenamiento
Restricciones	solo límites del actuador	explícitas, en cada paso	penalizadas en la recompensa o con un filtro externo
Coste en línea	despreciable	un problema de optimización por paso	una evaluación de la política
Adaptación a cambios	reajuste manual	cambiar el modelo o el coste	reentrenar
Interpretabilidad	alta	alta (el plan es visible)	baja
Cuándo elegirlo	lazos básicos, sin objetivo económico	objetivos económicos con buen modelo y previsiones	dinámicas difíciles de modelar con un simulador rico o decisiones discretas complejas

Cuadro 5.2: Comparación cualitativa de los tres enfoques.

Definición 5.3 (Modelo del reactor). Para una reacción de primer orden A $\to$ B en un reactor de volumen V refrigerado por una camisa a temperatura T_j:

$$\dot{C}_A = -k(T)C_A, \qquad \rho c_p V\,\dot{T} = Q_g(T, C_A) - Q_r(T),$$
$$Q_g = (-\Delta H)\,k(T)\,C_A V, \qquad Q_r = UA\,(T - T_j),$$

con $k(T) = k_{\text{ref}} \exp\!\big(-\frac{E}{R}(\frac{1}{T} - \frac{1}{T_{\text{ref}}})\big)$ (ley de Arrhenius).

Proposición 5.4 (Criterio de Semenov). *Sea C_A aproximadamente constante (fase inicial del lote, con conversión pequeña) y sea $\bar{T}$ un equilibrio de la ecuación de energía, $Q_g(\bar{T}) = UA(\bar{T} - T_j)$. El equilibrio es asintóticamente estable si $Q_g'(\bar{T}) < UA$ e inestable si $Q_g'(\bar{T}) > UA$. Además, $Q_g'(T) = Q_g(T)\,E/(RT^2)$.*

5.4 Gestión de una batería con precios horarios

El ejemplo 3.11 y el ejemplo 4.14 resuelven el mismo problema por dos vías. Con precios conocidos, el LP es exacto y rápido: 1,8842 € en el día del cuaderno. La programación dinámica sobre una rejilla de 81 niveles obtiene 1,8837 €, y su interés aparece cuando los precios son *inciertos*: la PD estocástica maximiza el beneficio esperado con el precio como parte del estado, y el RL la aproxima cuando el estado crece (varias baterías, una planta fotovoltaica, la degradación como estado). Tres decisiones de modelado dominan el resultado: el rendimiento de ida y vuelta (un ciclo solo compensa si el cociente de precios supera $1/\eta^2$), el coste de degradación por kWh cíclico y la condición terminal (sin $e_{24} = e_0$, el optimizador vacía la batería al final del día porque el modelo no ve el día siguiente).

5.5 Refrigeración de centros de datos: el caso publicado de Google y DeepMind

Lo que sigue usa exclusivamente la información publicada; los detalles internos del sistema no se han hecho públicos.

2016: recomendaciones. Según la publicación de DeepMind de julio de 2016 [11], un conjunto de redes neuronales profundas se entrenó con datos históricos de miles de sensores (temperaturas, potencias, velocidades de bombas, consignas) para predecir la eficacia en el uso de la energía (PUE, cociente entre la energía total del edificio y la consumida por los equipos informáticos) futura; otros dos conjuntos de redes predecían la temperatura y la presión de la hora siguiente para comprobar que las acciones recomendadas respetaban los límites de operación. En pruebas en un centro de datos en funcionamiento, alternando periodos con el sistema activado y desactivado, se publicó una reducción de hasta el 40 % de la energía de refrigeración, equivalente a un 15 % de reducción del exceso de PUE sobre la unidad.

2018: control autónomo. En agosto de 2018 [15] se publicó que el sistema pasó a controlar directamente la refrigeración: cada cinco minutos, un sistema de IA en la nube toma una instantánea de miles de sensores, predice con redes neuronales el efecto de combinaciones de acciones sobre el consumo futuro y elige las que lo minimizan respetando restricciones de seguridad; el sistema de control local verifica las acciones antes de aplicarlas. Entre las salvaguardas publicadas figuran la estimación de la incertidumbre (solo se aplican acciones de alta confianza), la verificación en dos capas, la vuelta automática a un estado neutro si se violan restricciones, reglas heurísticas de respaldo y la posibilidad de que los operadores salgan del modo de IA en cualquier momento. Se publicó un ahorro medio de alrededor del 30 % tras unos meses de operación.

Un estudio académico relacionado. Lazic *et al.* [23] describieron en NeurIPS 2018 un MPC con un modelo ARX disperso de historia de 5 pasos, ajustado por mínimos cuadrados recursivos tras unas 3 h de exploración aleatoria acotada, que regulaba temperaturas y presiones de un sector de servidores cada 30 s; la exploración limitaba el rango y la variación máxima de cada actuador entre pasos. En condiciones comparables, el coste del controlador fue de 116,7 frente a 142,2 de los controladores locales, y estimaron un ahorro de alrededor del 9 % en el coste de refrigeración de ese sector, con la advertencia de que harían falta experimentos más largos para confirmarlo.

El caso reúne casi todo el módulo: un modelo aprendido (tema 2), un optimizador sobre predicciones con restricciones (tema 3), un periodo de recomendación con humanos en el lazo antes del control autónomo, salvaguardas en capas y la medición del ahorro con experimentos de activación alterna (tema 6).

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
EnergyPlus [7]	simulación energética de edificios completa	libre	gemelo físico de un edificio
Biblioteca Buildings de Modelica [32]	componentes de climatización y control	libre	modelos de instalaciones a medida
BOPTEST [5]	bancos de prueba normalizados para controladores de edificios	libre	comparar controladores de forma reproducible
Sinergym [6]	EnergyPlus como entorno Gymnasium	libre (MIT)	entrenar agentes de climatización
CasADi, do-mpc [3, 12]	NMPC del reactor con restricciones de seguridad	libres (LGPL-3)	procesos químicos no lineales
CVXPY, Pyomo	programación lineal de baterías y despacho	libres	arbitraje y planificación energética

Cuadro 5.3: Tecnologías para los casos de estudio.

Tema 6

Puesta en producción en planta

Un controlador que funciona en el gemelo digital está a mitad de camino. En planta hay operadores que deben entender y poder anular lo que hace, alarmas que no deben saturarlos, un sistema de control básico y otro de seguridad que no se pueden comprometer, una red industrial que es un objetivo de ciberataques y un marco normativo. Este tema recorre esos requisitos. La arquitectura por capas sigue el capítulo 15 («Architecture and System Design») de Åström y Murray [4]; la gestión de alarmas, la norma ISA-18.2 [21]; la seguridad funcional, IEC 61511 [19]; la ciberseguridad, la guía NIST SP 800-82 [45] y la serie IEC 62443 [20]; la validación y el seguimiento de modelos, el curso CS329S de Stanford [18] y Sculley *et al.* [41], y la regulación, el Reglamento europeo de inteligencia artificial [40]. Las normas son documentos de pago: aquí se usa solo lo que sus organismos difunden públicamente.

6.1 Arquitectura por capas

La figura 6.1 resume el principio de diseño: cada capa trabaja en una escala de tiempo y confía en la inferior. El MPC o la política de RL *no* mueven válvulas: envían consignas a los PID existentes, que siguen funcionando si la capa superior falla. Entre ambas, un filtro valida cada consigna. El *sistema instrumentado de seguridad* (SIS), que detiene el proceso ante condiciones peligrosas, es independiente de todo lo anterior, con sus propios sensores y lógica, y se diseña según IEC 61511 [19], la aplicación al sector de procesos de la seguridad funcional de IEC 61508, que asigna a cada función de seguridad un nivel de integridad (SIL, de 1 a 4). Ningún componente de aprendizaje automático forma parte del SIS. El caso de Google [15] sigue el mismo esquema: la IA en la nube propone, el sistema local verifica y aplica, y las reglas clásicas quedan como respaldo.

6.2 Modos de operación y lazo humano

Definición 6.1 (Modos de operación). Se distinguen tres modos de creciente autonomía: *asesor* (el sistema recomienda y el operador decide y ejecuta), *supervisado* (el sistema ejecuta dentro de límites y el operador vigila y puede intervenir en cualquier momento) y *autónomo con supervisión* (el sistema opera con salvaguardas automáticas y el operador interviene por excepción). En todos ellos el operador puede volver al control básico.

Un despliegue prudente recorre los modos en orden y solo avanza cuando hay evidencia: el caso de los centros de datos pasó de recomendaciones en 2016 [11] a control autónomo con supervisión en 2018 [15]. El paso entre modos no debe provocar saltos en los actuadores.

Proposición 6.2 (Transferencia sin saltos). *Sea un PI en forma posicional, $u_k = K_c e_k + I_k$, que toma el control en el instante k_0 de un sistema que aplicaba u_{k_0-1}. Si se inicializa $I_{k_0} = u_{k_0-1} - K_c e_{k_0}$, entonces $u_{k_0} = u_{k_0-1}$.*

Con la zona del ejemplo 1.15, si la capa supervisora aplicaba 1,2 kW y el error al recuperar el PI es de 0,4 K, el integrador se inicializa en $1{,}2 - 1{,}429 \cdot 0{,}4 = 0{,}629$ kW. El mismo principio se aplica al pasar del PI al MPC: el MPC debe penalizar los cambios de la acción respecto a la última aplicada.

6.3 Alarmas

La norma ANSI/ISA-18.2, base de la internacional IEC 62682 [21], organiza la gestión de alarmas en un ciclo de vida (filosofía, identificación, racionalización, diseño, implantación, operación, mantenimiento, seguimiento, gestión del cambio y auditoría) y da referencias de carga: alrededor de 150 alarmas al día (una cada 10 minutos de media) se considera muy probablemente aceptable y alrededor de 300 al día (dos cada 10 minutos), el máximo manejable. Una alarma solo debe existir si exige una acción del operador. Una capa de IA añade fuentes nuevas que hay que racionalizar con el mismo criterio: residuos del modelo fuera de banda, proximidad a restricciones, salidas a modo de respaldo, pérdida de comunicación con la capa supervisora o predicciones con incertidumbre alta.

6.4 Validación y despliegue

Definición 6.3 (Etapas de validación). (i) *Validación fuera de línea*: el controlador se evalúa en el gemelo y sobre históricos, con escenarios no usados en su diseño. (ii) *Modo sombra*: el controlador calcula sus decisiones con datos en tiempo real, pero no las aplica; se comparan con las del sistema existente y se comprueba que serían seguras. (iii) *Experimento en planta*: se alterna el controlador nuevo y el existente por periodos (días o semanas) y se mide la diferencia con un diseño de comparación pareada [33, cap. 2]. (iv) *Despliegue con seguimiento*: se vigilan los residuos del modelo, la distribución de las entradas y los indicadores, con criterios de vuelta atrás definidos de antemano.

Los modelos degradan: cambian los equipos, el uso del edificio o el producto, y las relaciones aprendidas dejan de valer. Sculley *et al.* [41] describen cómo un sistema de aprendizaje automático acumula «deuda técnica» por dependencias de datos ocultas, bucles de realimentación (el controlador cambia los datos con los que se reentrenará) y configuración, y las clases 10 y 11 de CS329S [18] tratan la detección de cambios de distribución y el seguimiento. En control, la señal de seguimiento más útil es el residuo del modelo en simulación libre sobre ventanas recientes: un sesgo sostenido indica que hay que recalibrar el gemelo.

6.5 El filtro de seguridad

Definición 6.5 (Filtro de seguridad). Sea un modelo $x_{k+1} = f(x_k, u_k, w_k)$ con perturbación acotada $w_k \in W$ y un conjunto seguro $\mathcal{S}_{\text{seg}}$. Para cada x se define el conjunto de acciones seguras $\mathbb{U}_{\text{seg}}(x) = \{u \in \mathbb{U} : \ f(x,u,w) \in \mathcal{S}_{\text{seg}} \ \forall w \in W\}$. El *filtro de seguridad* sustituye la acción propuesta u^{p} por su proyección $u = \text{proy}_{\mathbb{U}_{\text{seg}}(x)}(u^{\text{p}})$.

Proposición 6.6 (Invariancia bajo el filtro). *Si $\mathbb{U}_{\text{seg}}(x) \neq \emptyset$ para todo $x \in \mathcal{S}_{\text{seg}}$ y $x_0 \in \mathcal{S}_{\text{seg}}$, entonces, cualquiera que sea la política que propone las acciones, $x_k \in \mathcal{S}_{\text{seg}}$ para todo k y toda secuencia de perturbaciones en W.*

La garantía es tan buena como el modelo y la cota de las perturbaciones: el filtro usa un modelo sencillo, conservador y verificable, no la red del agente. Cuando $\mathbb{U}_{\text{seg}}$ es un intervalo, la proyección es un simple recorte. Se completa con límites de velocidad de cambio (Lazic *et al.* [23] limitaban la variación de cada actuador entre pasos), un *perro guardián* (*watchdog*) que devuelve el control a los PID si la capa supervisora deja de enviar consignas válidas, y la transferencia sin saltos de la proposición 6.2.

6.6 Ciberseguridad de la tecnología de operación

Los sistemas de control industrial (tecnología de operación, OT) priorizan la disponibilidad y la seguridad física por encima de la confidencialidad, funcionan durante décadas con equipos difíciles de actualizar y usan protocolos diseñados sin autenticación. La guía NIST SP 800-82, revisión 3, de 2023 [45], adapta la gestión de riesgos de ciberseguridad a este entorno. La serie IEC 62443 [20], desarrollada por IEC e ISA, se organiza en cuatro bloques (general; políticas y procedimientos; sistema; componentes) y aporta dos conceptos centrales: la partición del sistema en *zonas* de activos con requisitos de seguridad comunes, comunicadas solo por *conductos* definidos, y los *niveles de seguridad* SL 1 a SL 4, ligados a la capacidad del adversario que se quiere resistir, desglosados en siete requisitos fundamentales (identificación y autenticación, control de uso, integridad del sistema, confidencialidad de los datos, flujo de datos restringido, respuesta oportuna a eventos y disponibilidad de recursos).

Para una capa de IA, de ahí se derivan reglas prácticas: (i) la capa supervisora vive en su propia zona y solo escribe consignas por un conducto autenticado y registrado, nunca acceso directo a los controladores; (ii) el sistema local valida cada consigna (el filtro de seguridad es también una defensa: limita el daño de una consigna maliciosa); (iii) si el modelo se ejecuta en la nube, la planta debe poder operar sin ella; (iv) los datos de entrenamiento son un activo: un atacante que los manipule puede inducir un modelo que se comporte mal en condiciones concretas, y (v) actualizar un modelo es un cambio en el sistema de control y pasa por la gestión del cambio.

6.7 Normas y regulación

Los sistemas de alto riesgo del Reglamento de IA deben cumplir los requisitos de la sección 2 de su capítulo III (artículos 8 a 15), entre ellos la gestión de riesgos, la gobernanza de los datos, el registro de eventos, la supervisión humana (artículo 14: el sistema debe diseñarse para que las personas puedan entender sus límites, detectar anomalías, no confiar en exceso en sus salidas, anular sus decisiones y detenerlo de forma segura) y la precisión, robustez y ciberseguridad. Un optimizador de climatización de oficinas no suele ser un componente de seguridad; el control de una red de calefacción urbana o de una subestación puede serlo. La clasificación de cada caso es una cuestión jurídica que debe evaluarse con especialistas; lo que el módulo aporta es que la arquitectura de la figura 6.1 facilita cumplir esos requisitos.

Referencia	Ámbito	Qué implica para una capa de IA
IEC 61508 / IEC 61511 [19]	seguridad funcional; sistemas instrumentados de seguridad en procesos	la IA queda fuera del SIS; el análisis de riesgos debe considerar sus modos de fallo
ANSI/ISA-18.2, IEC 62682 [21]	gestión de alarmas	racionalizar las alarmas nuevas; vigilar la carga del operador
IEC 62443 [20], NIST SP 800-82 [45]	ciberseguridad de sistemas de automatización	zonas y conductos; autenticación y registro de consignas
Reglamento (UE) 2024/1689 [40]	inteligencia artificial	los sistemas destinados a ser componentes de seguridad en la gestión y operación de infraestructuras críticas, como el suministro de agua, gas, calefacción o electricidad, se clasifican como de alto riesgo (anexo III, punto 2)

Cuadro 6.1: Marco normativo de referencia.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
OPC UA	comunicación industrial con seguridad (autenticación, cifrado)	estándar abierto	leer medidas y escribir consignas desde la capa supervisora
Historiadores de proceso	almacenamiento de series temporales de planta	comercial o libre	datos de identificación y de seguimiento
MLflow u otros registros de modelos	versionado de modelos, datos y métricas	libre (Apache-2.0)	trazabilidad de qué modelo controló la planta y cuándo
Contenedores y orquestación	despliegue reproducible de la capa supervisora	libre	separar entornos de prueba y producción
Evidently, WhyLogs	seguimiento de distribuciones y de la deriva de los datos	libres	alarmas de degradación del modelo

Cuadro 6.2: Tecnologías para la puesta en producción.

Bibliografía

[1] J. Achiam, D. Held, A. Tamar y P. Abbeel. Constrained policy optimization. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR 70, 2017. https://proceedings.mlr.press/v70/achiam17a.html

[2] E. Altman. *Constrained Markov Decision Processes*. Chapman & Hall/CRC, 1999.

[3] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings y M. Diehl. CasADi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019. https://web.casadi.org/

[4] K. J. Åström y R. M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*, 2.ª ed. Princeton University Press, 2021. Caps. 3 «System Modeling», 5 «Dynamic Behavior», 6 «Linear Systems», 11 «PID Control» y 15 «Architecture and System Design». Versión electrónica libre. https://fbsbook.org/

[5] D. Blum, J. Arroyo, S. Huang, J. Drgoňa, F. Jorissen, H. T. Walnum, Y. Chen, K. Benne, D. Vrabie, M. Wetter y L. Helsen. Building optimization testing framework (BOPTEST) for simulation-based benchmarking of control strategies in buildings. *Journal of Building Performance Simulation*, 14:586–610, 2021.

[6] A. Campoy-Nieves, A. Manjavacas, J. Jiménez-Raboso, M. Molina-Solana y J. Gómez-Romero. Sinergym – A virtual testbed for building energy optimization with reinforcement learning. Prepublicación, 2024. https://arxiv.org/abs/2412.08293

[7] D. B. Crawley, L. K. Lawrie, F. C. Winkelmann, W. F. Buhl, Y. J. Huang, C. O. Pedersen, R. K. Strand, R. J. Liesen, D. E. Fisher y M. J. Witte. EnergyPlus: creating a new-generation building energy simulation program. *Energy and Buildings*, 33(4):319–331, 2001. https://energyplus.net/

[8] S. Diamond y S. Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016. https://www.cvxpy.org/

[9] J. Drgoňa, J. Arroyo, I. Cupeiro Figueroa, D. Blum, K. Arendt, D. Kim, E. Perarnau Ollé, J. Oravec, M. Wetter, D. L. Vrabie y L. Helsen. All you need to know about model predictive control for buildings. *Annual Reviews in Control*, 2020. https://doi.org/10.1016/j.arcontrol.2020.09.001

[10] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal y T. Hester. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110, 2021. https://arxiv.org/abs/2003.11881

[11] R. Evans y J. Gao. DeepMind AI reduces Google data centre cooling bill by 40%. Blog de Google DeepMind, 20 de julio de 2016. https://deepmind.google/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/

[12] F. Fiedler, B. Karg, L. Lüken, D. Brandner, M. Heinlein, F. Brabender y S. Lucia. do-mpc: Towards FAIR nonlinear and robust model predictive control. *Control Engineering Practice*, 140:105676, 2023. https://www.do-mpc.com/

[13] S. Fujimoto, H. van Hoof y D. Meger. Addressing function approximation error in actor-critic methods. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR 80:1587–1596, 2018. https://arxiv.org/abs/1802.09477

[14] S. Fuller, B. Greiner, J. Moore, R. M. Murray, R. van Paassen y R. Yorke. The Python Control Systems Library (python-control). En *IEEE Conference on Decision and Control (CDC)*, 2021. https://python-control.readthedocs.io/

[15] C. Gamble y J. Gao. Safety-first AI for autonomous data centre cooling and industrial control. Blog de Google DeepMind, 17 de agosto de 2018. https://deepmind.google/blog/safety-first-ai-for-autonomous-data-centre-cooling-and-industrial-control/

[16] T. Haarnoja, A. Zhou, P. Abbeel y S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. En *International Conference on Machine Learning (ICML)*, 2018. https://arxiv.org/abs/1801.01290

[17] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel y S. Levine. Soft actor-critic algorithms and applications. Prepublicación, 2018. https://arxiv.org/abs/1812.05905

[18] C. Huyen. CS329S: Machine Learning Systems Design. Stanford University, invierno de 2022. Clases 8 «Deployment», 10 «Diagnosis of ML system failures & data distribution shifts & monitoring» y 11 «Monitoring & Continual Learning». https://stanford-cs329s.github.io/

[19] International Electrotechnical Commission. IEC 61511-1:2016. *Functional safety – Safety instrumented systems for the process industry sector – Part 1: Framework, definitions, system, hardware and application programming requirements*. IEC, 2016. https://webstore.iec.ch/publication/24241

[20] International Electrotechnical Commission e ISA. Serie IEC 62443, *Security for industrial automation and control systems*. Resumen público del comité de sistemas de energía inteligente de IEC. https://syc-se.iec.ch/deliveries/cybersecurity-guidelines/security-standards-and-best-practices/iec-62443/

[21] International Society of Automation. ANSI/ISA-18.2, *Management of Alarm Systems for the Process Industries*, 2009, base de la norma internacional IEC 62682. Resumen público de su ciclo de vida y de sus referencias de carga en: «Implementing alarm management per the ANSI/ISA-18.2 standard», *Control Engineering*. https://www.controleng.com/articles/implementing-alarm-management-per-the-ansi-isa-18-2-standard/

[22] W. Kritzinger, M. Karner, G. Traar, J. Henjes y W. Sihn. Digital Twin in manufacturing: A categorical literature review and classification. *IFAC-PapersOnLine*, 51(11):1016–1022, 2018.

[23] N. Lazic, T. Lu, C. Boutilier, M. K. Ryu, E. J. Wong, B. Roy y G. Imwalle. Data center cooling using model-predictive control. En *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 3818–3827, 2018. https://research.google/pubs/data-center-cooling-using-model-predictive-control/

[24] S. Levine. CS285: Deep Reinforcement Learning. University of California, Berkeley, otoño de 2023. Clases 7 «Value Function Methods», 8 «Deep RL with Q-Functions», 10 «Optimal Control and Planning» y 15 y 16 «Offline Reinforcement Learning». https://rail.eecs.berkeley.edu/deeprlcourse-fa23/

[25] S. Levine y equipo docente de CS285. CS185/285: Deep Reinforcement Learning, Decision Making, and Control. Tarea 3, «Q-Learning and Actor-Critic Algorithms». University of California, Berkeley, primavera de 2026. §2.5 (doble Q-learning) y §3 (*soft actor-critic*). https://rail.eecs.berkeley.edu/deeprlcourse/static/homeworks/hw3.pdf

[26] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver y D. Wierstra. Continuous control with deep reinforcement learning. En *International Conference on Learning Representations (ICLR)*, 2016. https://arxiv.org/abs/1509.02971

[27] L. Ljung. *System Identification: Theory for the User*, 2.ª ed. Prentice Hall, 1999. Texto del curso 6.435 del MIT.

[28] H. Mania, A. Guy y B. Recht. Simple random search of static linear policies is competitive for reinforcement learning. En *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018. https://arxiv.org/abs/1803.07055

[29] M. A. Dahleh. 6.435: System Identification. Massachusetts Institute of Technology, MIT OpenCourseWare, primavera de 2005. Temario y hoja de problemas 5. https://ocw.mit.edu/courses/6-435-system-identification-spring-2005/

[30] B. S. Johnston. 10.450: Process Dynamics, Operations, and Control. Massachusetts Institute of Technology, MIT OpenCourseWare, primavera de 2006. Lecciones 5 «The Heated Tank» y 6 «The Exothermic Reactor». https://ocw.mit.edu/courses/10-450-process-dynamics-operations-and-control-spring-2006/

[31] Modelica Association. *Functional Mock-up Interface (FMI)*, versión 3.0. https://fmi-standard.org/

[32] Modelica Association. *Modelica Language* y bibliotecas asociadas. https://modelica.org/

[33] D. C. Montgomery. *Design and Analysis of Experiments*, 9.ª ed. Wiley, 2017. Cap. 2 «Simple Comparative Experiments», §2.5 sobre comparaciones pareadas.

[34] K. S. Narendra y K. Parthasarathy. Identification and control of dynamical systems using neural networks. *IEEE Transactions on Neural Networks*, 1(1):4–27, 1990.

[35] A. Y. Ng, D. Harada y S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. En *International Conference on Machine Learning (ICML)*, 278–287, 1999.

[36] J. Nocedal y S. J. Wright. *Numerical Optimization*, 2.ª ed. Springer, 2006. Cap. 17 «Penalty and Augmented Lagrangian Methods», §17.2 y teorema 17.3.

[37] X. B. Peng, M. Andrychowicz, W. Zaremba y P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. En *IEEE International Conference on Robotics and Automation (ICRA)*, 2018. https://arxiv.org/abs/1710.06537

[38] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus y N. Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[39] J. B. Rawlings, D. Q. Mayne y M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*, 2.ª ed. Nob Hill Publishing, 2017. Caps. 1 «Getting Started with Model Predictive Control», 2 «Model Predictive Control: Regulation» y 8 «Numerical Optimal Control». Versión electrónica libre. https://sites.engineering.ucsb.edu/~jbraw/mpc/

[40] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2024/1689, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial. Capítulo III, sección 2 (artículos 8 a 15) y anexo III. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[41] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo y D. Dennison. Hidden technical debt in machine learning systems. En *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2503–2511, 2015.

[42] D. E. Seborg, T. F. Edgar, D. A. Mellichamp y F. J. Doyle III. *Process Dynamics and Control*, 4.ª ed. Wiley, 2016.

[43] S. Skogestad. Simple analytic rules for model reduction and PID controller tuning. *Journal of Process Control*, 13:291–309, 2003.

[44] Stanford University. EE363: Linear Dynamical Systems. Primavera de 2026. Clases 16 «Linear quadratic regulator: Discrete-time finite horizon», 17 «LQR via Lagrange multipliers» y 18 «Infinite horizon linear quadratic regulator». https://ee363.stanford.edu/lectures.html

[45] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule y M. Thompson. *Guide to Operational Technology (OT) Security*. NIST Special Publication 800-82, revisión 3. National Institute of Standards and Technology, septiembre de 2023. https://csrc.nist.gov/pubs/sp/800/82/r3/final

[46] B. Stellato, G. Banjac, P. Goulart, A. Bemporad y S. Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

[47] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*, 2.ª ed. MIT Press, 2018. Cap. 3, ejercicios 3.15 y 3.16; §6.7 «Maximization Bias and Double Learning»; cap. 13 «Policy Gradient Methods». http://incompleteideas.net/book/the-book-2nd.html

[48] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba y P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. En *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017. https://arxiv.org/abs/1703.06907

[49] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis *et al.* Gymnasium: A standard interface for reinforcement learning environments. Prepublicación, 2024. https://arxiv.org/abs/2407.17032

[50] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen y M. Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021. https://docs.acados.org/

[51] Z. Wang y T. Hong. Reinforcement learning for building controls: The opportunities and challenges. *Applied Energy*, 269:115036, 2020.

[52] J. G. Ziegler y N. B. Nichols. Optimum settings for automatic controllers. *Transactions of the ASME*, 64:759–768, 1942.